AGLC Law Firm CRM: Configuration Guide

Project: AGLC Law Firm CRM
Version: 1.0
Date: March 12, 2026
Purpose: System configuration and environment setup

Table of Contents

1. Development Environment
2. Environment Variables
3. Database Configuration
4. OAuth Configuration
5. External Services
6. Logging Configuration
7. Monitoring Configuration
8. Security Configuration

Development Environment

Prerequisites

Ensure you have the following installed:

- Node.js 22+ (Check with `node --version`)
- PostgreSQL 14+ (Check with `psql --version`)
- Git (Check with `git --version`)

- pnpm 10+ (Check with `pnpm --version`)

Initial Setup

```
# Clone the repository
git clone https://github.com/yourorg/aglc-crm.git
cd aglc-crm

# Install dependencies
pnpm install

# Create .env.local file
cp .env.example .env.local

# Create database
createdb aglc_crm

# Run migrations
pnpm db:push

# Start development server
pnpm dev
```

Verify Setup

```
# Check Node version
node --version  # Should be 22.x.x

# Check pnpm version
pnpm --version  # Should be 10.x.x

# Check database connection
psql -U postgres -d aglc_crm -c "SELECT 1"

# Check development server
curl http://localhost:3000/api/health
```

Environment Variables

Development Environment (.env.local)

Create a `.env.local` file in the project root:

```
# Database
DATABASE_URL=postgresql://postgres:password@localhost:5432/aglc_crm

# OAuth (Manus)
VITE_APP_ID=your_app_id
OAUTH_SERVER_URL=https://api.manus.im
VITE_OAUTH_PORTAL_URL=https://portal.manus.im
JWT_SECRET=your_jwt_secret_key_min_32_chars

# LLM (OpenAI)
OPENAI_API_KEY=sk-your-openai-key
OPENAI_MODEL=gpt-4

# Twilio (WhatsApp)
TWILIO_ACCOUNT_SID=ACxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
TWILIO_AUTH_TOKEN=your_twilio_auth_token
TWILIO_WHATSAPP_NUMBER=+1234567890

# SendGrid (Email)
SENDGRID_API_KEY=SG.your_sendgrid_key

# AWS S3 (File Storage)
AWS_ACCESS_KEY_ID=your_aws_access_key
AWS_SECRET_ACCESS_KEY=your_aws_secret_key
AWS_S3_BUCKET=aglc-crm-bucket
AWS_S3_REGION=us-east-1

# Redis (Caching & Sessions)
REDIS_URL=redis://localhost:6379

# Logging
LOG_LEVEL=debug

# Application
NODE_ENV=development
PORT=3000
VITE_API_URL=http://localhost:3000
```

Production Environment

Set these variables in the Manus UI or your deployment platform:

```
# Database (use managed database)
DATABASE_URL=postgresql://user:pass@host:5432/aglc_crm

# OAuth
VITE_APP_ID=production_app_id
OAUTH_SERVER_URL=https://api.manus.im
VITE_OAUTH_PORTAL_URL=https://portal.manus.im
JWT_SECRET=production_jwt_secret_min_32_chars

# LLM
OPENAI_API_KEY=sk-production-key

# Twilio
TWILIO_ACCOUNT_SID=production_account_sid
TWILIO_AUTH_TOKEN=production_auth_token
TWILIO_WHATSAPP_NUMBER=+1234567890

# SendGrid
SENDGRID_API_KEY=SG.production_key

# AWS S3
AWS_ACCESS_KEY_ID=production_access_key
AWS_SECRET_ACCESS_KEY=production_secret_key
AWS_S3_BUCKET=aglc-crm-production
AWS_S3_REGION=us-east-1

# Redis
REDIS_URL=redis://production-host:6379

# Logging
LOG_LEVEL=info

# Application
NODE_ENV=production
PORT=3000
VITE_API_URL=https://aglc-crm.example.com
```

Environment Variable Validation

```
// server/_core/env.ts
import { z } from "zod";

const envSchema = z.object({
  DATABASE_URL: z.string().url(),
  VITE_APP_ID: z.string(),
  OAUTH_SERVER_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
  OPENAI_API_KEY: z.string().startsWith("sk-"),
  NODE_ENV: z.enum(["development", "production"]),
  PORT: z.string().default("3000"),
});

export const ENV = envSchema.parse(process.env);
```

Database Configuration

PostgreSQL Setup

Create Database

```
# Connect to PostgreSQL
psql -U postgres

# Create database
CREATE DATABASE aglc_crm;

# Create user (optional)
CREATE USER aglc_user WITH PASSWORD 'secure_password';
GRANT ALL PRIVILEGES ON DATABASE aglc_crm TO aglc_user;

# Exit psql
\q
```

Connection String

```
postgresql://username:password@localhost:5432/aglc_crm
```

Connection Pooling

For production, use connection pooling:

```
# Install pgBouncer
brew install pgbouncer  # macOS
apt-get install pgbouncer  # Ubuntu

# Configure pgbouncer.ini
[databases]
aglc_crm = host=localhost port=5432 dbname=aglc_crm

[pgbouncer]
listen_port = 6432
max_client_conn = 1000
default_pool_size = 25
```

Drizzle ORM Configuration

File: `drizzle.config.ts`

```
import type { Config } from "drizzle-kit";

export default {
  schema: "./drizzle/schema.ts",
  out: "./drizzle/migrations",
  driver: "mysql2",
  dbCredentials: {
    connectionString: process.env.DATABASE_URL!,
  },
  verbose: true,
  strict: true,
} satisfies Config;
```

Database Migrations

```
# Generate migrations
pnpm db:generate

# Apply migrations
pnpm db:push

# View migration status
pnpm db:status

# Rollback last migration
pnpm db:rollback
```

Backup & Restore

```
# Backup database
pg_dump aglc_crm > backup.sql

# Restore database
psql aglc_crm < backup.sql

# Backup with compression
pg_dump -Fc aglc_crm > backup.dump

# Restore compressed backup
pg_restore -d aglc_crm backup.dump
```

OAuth Configuration

Manus OAuth Setup

Register Application

1. Go to [Manus Developer Portal]
2. Create a new application

3. Set redirect URI: `http://localhost:3000/api/oauth/callback`
4. Copy Application ID and Secret

Environment Variables

```
VITE_APP_ID=your_application_id
OAUTH_SERVER_URL=https://api.manus.im
VITE_OAUTH_PORTAL_URL=https://portal.manus.im
JWT_SECRET=your_jwt_secret_key_min_32_chars
```

OAuth Flow

```
// server/_core/oauth.ts
import { OAuth2Client } from "google-auth-library";

const oauth2Client = new OAuth2Client(
  process.env.VITE_APP_ID,
  process.env.OAUTH_SERVER_URL,
  "http://localhost:3000/api/oauth/callback"
);

// Generate login URL
const loginUrl = oauth2Client.generateAuthUrl({
  access_type: "offline",
  scope: ["openid", "profile", "email"],
});

// Handle callback
async function handleOAuthCallback(code: string) {
  const { tokens } = await oauth2Client.getToken(code);
  const ticket = await oauth2Client.verifyIdToken({
    idToken: tokens.id_token,
  });
  return ticket.getPayload();
}
```

External Services

OpenAI LLM Configuration

API Key Setup

1. Go to [OpenAI Platform]
2. Create API key
3. Set environment variable: `OPENAI_API_KEY=sk-...`

Configuration

```
// server/_core/llm.ts
import OpenAI from "openai";

const openai = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY,
});

export async function invokeLLM(params: {
  messages: Array<{ role: string; content: string }>;
  model?: string;
  temperature?: number;
  max_tokens?: number;
}) {
  return openai.chat.completions.create({
    model: params.model || "gpt-4",
    messages: params.messages,
    temperature: params.temperature || 0.7,
    max_tokens: params.max_tokens || 2000,
  });
}
```

Rate Limiting

```
// Implement rate limiting to avoid excessive costs
const rateLimiter = new Map<string, number[]>();

function checkRateLimit(userId: string, limit = 10, window = 3600000) {
  const now = Date.now();
  const userRequests = rateLimiter.get(userId) || [];
  const recentRequests = userRequests.filter(t => now - t < window);

  if (recentRequests.length >= limit) {
    throw new Error("Rate limit exceeded");
  }

  recentRequests.push(now);
  rateLimiter.set(userId, recentRequests);
}
```

Twilio WhatsApp Configuration

API Credentials

1. Go to [Twilio Console]
2. Create WhatsApp sender
3. Set environment variables:
   - `TWILIO_ACCOUNT_SID`
   - `TWILIO_AUTH_TOKEN`
   - `TWILIO_WHATSAPP_NUMBER`

Configuration

```
// server/_core/twilio.ts
import twilio from "twilio";

const client = twilio(
  process.env.TWILIO_ACCOUNT_SID,
  process.env.TWILIO_AUTH_TOKEN
);

export async function sendWhatsAppMessage(
  to: string,
  body: string
) {
  return client.messages.create({
    from: `whatsapp:${process.env.TWILIO_WHATSAPP_NUMBER}`,
    to: `whatsapp:${to}`,
    body: body,
  });
}
```

Webhook Setup

```
# Set webhook URL in Twilio Console
https://your-domain.com/api/webhooks/whatsapp
```

SendGrid Email Configuration

API Key Setup

1. Go to [SendGrid Console]
2. Create API key
3. Set environment variable: `SENDGRID_API_KEY=SG...`

Configuration

```
// server/_core/sendgrid.ts
import sgMail from "@sendgrid/mail";

sgMail.setApiKey(process.env.SENDGRID_API_KEY!);

export async function sendEmail(params: {
  to: string;
  from: string;
  subject: string;
  html: string;
}) {
  return sgMail.send({
    to: params.to,
    from: params.from,
    subject: params.subject,
    html: params.html,
  });
}
```

AWS S3 Configuration

Credentials Setup

1. Go to [AWS Console]
2. Create S3 bucket
3. Create IAM user with S3 access
4. Set environment variables:
    - `AWS_ACCESS_KEY_ID`
    - `AWS_SECRET_ACCESS_KEY`
    - `AWS_S3_BUCKET`
    - `AWS_S3_REGION`

Configuration

```
// server/storage.ts
import { S3Client, PutObjectCommand } from "@aws-sdk/client-s3";

const s3Client = new S3Client({
  region: process.env.AWS_S3_REGION,
  credentials: {
    accessKeyId: process.env.AWS_ACCESS_KEY_ID!,
    secretAccessKey: process.env.AWS_SECRET_ACCESS_KEY!,
  },
});

export async function storagePut(
  key: string,
  body: Buffer,
  contentType: string
) {
  const command = new PutObjectCommand({
    Bucket: process.env.AWS_S3_BUCKET,
    Key: key,
    Body: body,
    ContentType: contentType,
  });

  await s3Client.send(command);

  return {
    key,
    url:
`https://${process.env.AWS_S3_BUCKET}.s3.${process.env.AWS_S3_REGION}.amazonaw
  };
}
```

Logging Configuration

Winston Logger Setup

```
// server/_core/logger.ts
import winston from "winston";

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || "info",
  format: winston.format.json(),
  transports: [
    new winston.transports.File({ filename: "error.log", level: "error" }),
    new winston.transports.File({ filename: "combined.log" }),
  ],
});

if (process.env.NODE_ENV !== "production") {
  logger.add(
    new winston.transports.Console({
      format: winston.format.simple(),
    })
  );
}

export default logger;
```

Structured Logging

```
// Log with context
logger.info("Case created", {
  caseId: 123,
  clientId: 456,
  userId: 789,
  timestamp: new Date(),
});

// Log errors
logger.error("Failed to create case", {
  error: error.message,
  stack: error.stack,
  userId: 789,
});
```

Monitoring Configuration

Sentry Error Tracking

Setup

```
npm install @sentry/node
```

Configuration

```
// server/_core/index.ts
import * as Sentry from "@sentry/node";

Sentry.init({
  dsn: process.env.SENTRY_DSN,
  environment: process.env.NODE_ENV,
  tracesSampleRate: 1.0,
});

// Attach to Express
app.use(Sentry.Handlers.requestHandler());
app.use(Sentry.Handlers.errorHandler());
```

Health Check Endpoint

```
// server/_core/index.ts
app.get("/api/health", async (req, res) => {
  const health = {
    status: "ok",
    timestamp: new Date(),
    uptime: process.uptime(),
    database: await checkDatabase(),
    redis: await checkRedis(),
  };

  res.json(health);
});

async function checkDatabase() {
  try {
    const db = await getDb();
    await db.query("SELECT 1");
    return { status: "ok" };
  } catch (error) {
    return { status: "error", error: error.message };
  }
}
```

Security Configuration

CORS Configuration

```
// server/_core/index.ts
import cors from "cors";

app.use(
  cors({
    origin: process.env.FRONTEND_URL || "http://localhost:5173",
    credentials: true,
    methods: ["GET", "POST", "PUT", "DELETE"],
    allowedHeaders: ["Content-Type", "Authorization"],
  })
);
```

Security Headers

```
// server/_core/index.ts
import helmet from "helmet";

app.use(helmet());

// Additional headers
app.use((req, res, next) => {
  res.setHeader("X-Content-Type-Options", "nosniff");
  res.setHeader("X-Frame-Options", "DENY");
  res.setHeader("X-XSS-Protection", "1; mode=block");
  res.setHeader("Strict-Transport-Security", "max-age=31536000");
  next();
});
```

Rate Limiting

```
// server/_core/index.ts
import rateLimit from "express-rate-limit";

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // limit each IP to 100 requests per windowMs
  message: "Too many requests from this IP",
});

app.use("/api/", limiter);
```

Environment-Specific Configuration

```
// server/_core/config.ts
export const config = {
  development: {
    debug: true,
    logLevel: "debug",
    corsOrigin: "http://localhost:5173",
  },
  production: {
    debug: false,
    logLevel: "info",
    corsOrigin: process.env.FRONTEND_URL,
  },
}[process.env.NODE_ENV || "development"];
```

Verification Checklist

- [ ] All environment variables set
- [ ] Database connection verified
- [ ] OAuth credentials configured
- [ ] LLM API key working
- [ ] Twilio credentials configured

- [ ] SendGrid API key working
- [ ] AWS S3 access verified
- [ ] Logging configured
- [ ] Monitoring setup complete
- [ ] Security headers configured
- [ ] Rate limiting enabled
- [ ] Health check endpoint working

Document Version: 1.0
Last Updated: March 12, 2026
Maintained By: DevOps Team